

Design a Job Scheduler

Cron · Airflow · Sidekiq · Quartz · Temporal – "ఒక heap lo సమయాలు పెట్టి, సమయం వచ్చినది తీద్దాం" అని అందరూ మొదలుపెడతారు.

మూడు విరుపులు. మొదటిది – ఘాతీయ **backoff** సరిగ్గా అదే వృథా చేస్తుంది, కేవలం 22 రెట్లు ఎక్కువ సమయం తీసుకుని. రెండోది – **16.4% jobs ఒకేసారి రెండుసార్లు** నడుస్తాయి, మరియు heartbeat పెట్టినా **2.65% jobs పాత ఫలితాన్ని** పట్టుకుంటాయి. మూడోది – 6 గంటల downtime తర్వాత **సరైన ఒక్క విధానం అంటూ ఉండదు**.

ఈ doc ఎలా చదవాలి

పక్కన editor తెరిచి కలిసి రాయండి. ఇక్కడి ప్రతి output, ప్రతి శాతం నిజంగా `node` lo `run` చేసినదే.

DEEP DIVE 24 (CONNECTION POOL) చదివారా? – TOKEN మళ్ళీ వస్తుంది, కానీ వేరే కారణంతో

DD 24 §8 lo ఒక **token** చూశాం: ప్రతి checkout కి ఒక ప్రత్యేక గుర్తు, తద్వారా పాత timer సజీవ connection ని లాగకూడదు. ఇక్కడ §8 lo **అదే ఆకారం** మళ్ళీ వస్తుంది – కానీ అది ఒక పునరావృతం కాదు, అది ఒక వేరే సమస్యకే అదే సాధనం.

తేడా ఇది:

Pool lo మనం ఒక వస్తువుని కాపాడుతున్నాం. ఇక్కడ మనం ఒక ఫలితాన్ని కాపాడుతున్నాం.

Pool lo పాత timer ఒక connection ని లాగితే, ఆ తప్పు **వెంటనే** కనిపిస్తుంది (queries కలిసిపోతాయి). ఇక్కడ ఆగిపోయిన ఒక worker మేల్కొని తన **పాత ఫలితాన్ని** రాస్తే, అది **సరైన ఫలితాన్ని తుడిచేస్తుంది** – మరియు ఎవరికీ ఏమీ తెలియదు. §8 lo అది **2.65%** jobs కి జరిగింది.

మరియు ఇంకొక తేడా: pool lo లీక్ అయిన connection ని మనం వెనక్కి లాగగలం. ఇక్కడ ఆగిపోయిన worker **మన అడుపుల్ లేడు** – అతన్ని ఆపడం అసాధ్యం. కాబట్టి లక్ష్యం మారుతుంది: **రెండుసార్లు నడవకుండా ఆపడం కాదు, రెండో ఫలితం అంగీకరించబడకుండా ఆపడం.**

విషయ సూచిక (Table of Contents)

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. అడిగింది ఏమిటి – మరియు ఇది ఒక queue కంటే ఎలా వేరు?
2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

Part 2 – మొదటి విరుపు: అందరూ ఒకే క్షణంలో

3. Step – ఒక జాబితా, ఒక `tick`, ఒక `retry`
4. **మొదటి విరుపు** – ఘాతీయ backoff ఏమీ ఆదా చేయలేదు
5. Step – jitter · మరియు షెడ్యూల్‌నే పరచడం

Part 3 – రెండో విరుపు: ఈ పని ఎవరిది?

6. Step – lease · "ఇది నాది, ఇంత సమయం వరకు"
7. **రెండో విరుపు** – 16.4% jobs ఒకేసారి రెండుసార్లు
8. Step – heartbeat · మరియు **ఆగిపోయిన worker**

Part 4 – మూడో విరుపు: తప్పిపోయిన runs

9. Step – scheduler 6 గంటలు పడిపోయింది
10. **మూడో విరుపు** – సరైన ఒక్క విధానం అంటూ లేదు

11. Step – విధానాన్ని job యజమాని చెప్పాడు

Part 5 – పూర్తి system

12. Step – (job, ఏ క్లజానికీ) – ఒకే ఒక్క key

13. మొత్తం code · 2,000 ప్రయోగాలు · **mutation testing**

Part 6 – Interview lo

14. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి

15. నోటితో చెప్పాల్సిన English script

16. Follow-ups – DAGs, ప్రాధాన్యత, సమయ మండలాలు

17. ఏమి నేర్చుకున్నాం

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. అడిగింది ఏమిటి – మరియు ఇది ఒక queue కంటే ఎలా వేరు?

"Job scheduler design చెయ్యి" అంటే చాలామంది వెంటనే priority queue గురించి మాట్లాడతారు. అది అవసరమే, కానీ అది **సులభమైన భాగం**. అసలు కష్టం వేరే చోట ఉంది, మరియు దాన్ని ఒక్క వాక్యంలో చెప్పొచ్చు:

QUEUE కీ SCHEDULER కీ అసలు తేడా

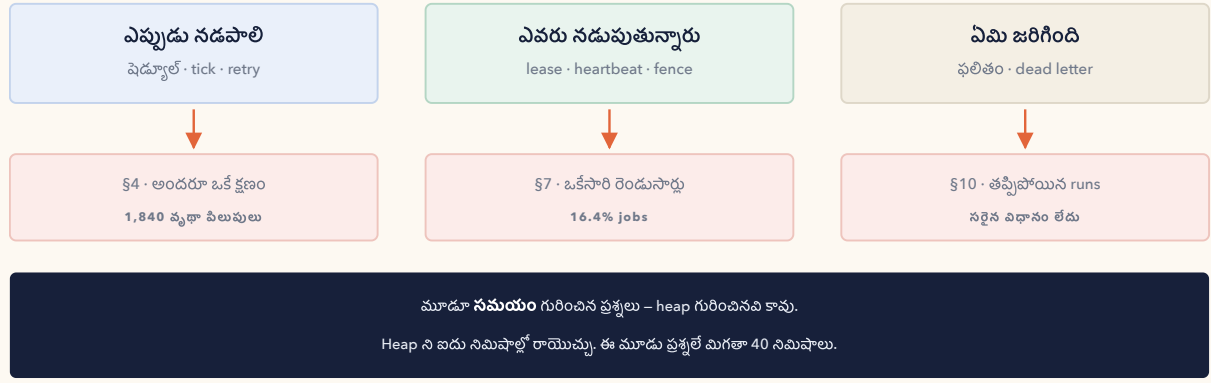
Queue lo ఒక పని ఒకసారి వస్తుంది. Scheduler lo ఒక పని పదేపదే వస్తుంది – మరియు అది ఎప్పుడు రావాలో మనం ముందే చెప్పాం.

అందుకే scheduler కి queue కి లేని మూడు ప్రశ్నలు ఉంటాయి:

- ఒక run విఫలమైతే **ఎప్పుడు** మళ్ళీ ప్రయత్నించాలి? (§4)
- ఒక worker పని మధ్యలో మాయమైతే ఆ పని **ఎవరిది**? (§7)
- మనం పడిపోయినప్పుడు తప్పిపోయిన runs ని **ఏమి చెయ్యాలి**? (§10)

ఈ మూడూ **సమయం గురించిన ప్రశ్నలు**, డేటా నిర్మాణం గురించినవి కావు. అందుకే ఈ doc lo heap గురించి ఒక్క §3 మాత్రమే ఉంది, మిగతాదంతా ఈ మూడు ప్రశ్నల గురించే.

మనం design చేస్తున్నది: పదేపదే జరగాల్సిన పనులను, **విఫలమయ్యే workers** మీద, **పడిపోయే scheduler** తో నడిపే వ్యవస్థ – మరియు అది ఆ మూడు వైఫల్యాల మధ్య కూడా **సరైన ఫలితం** ఇవ్వాలి.



2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

ప్రశ్న	ఎందుకు అడుగుతున్నాం
ఒక job ఖచ్చితంగా ఒకసారి నడవాలా, లేదా కనసం ఒకసారి చాలా?	ఖచ్చితంగా ఒకసారి అసాధ్యం . §8 lo ఎందుకీ చూపిస్తాం. ఇది ఈ మొత్తం design ని నిర్ణయిస్తుంది
Job idempotent దా?	కాకపోతే retry ప్రమాదకరం. Idempotent కాని jobs కి §12 lo key అవసరం
ఒక run ఎంతసేపు పడుతుంది? గరిష్ఠం ఎంత?	ఇదే lease వ్యవధిని నిర్ణయిస్తుంది. §7 lo తప్పు lease 16.4% duplicates ఇస్తుంది
Scheduler పడిపోతే తప్పిపోయిన runs ని ఏమి చెయ్యాలి?	దీనికి ఒక సమాధానం లేదు – §10
Workers ఎన్ని? ఒకే machine యా, చాలానా?	ఒకటే అయితే lease అవసరం లేదు. చాలా అయితే §6 మొత్తం అవసరం
సమయ మండలం (timezone) ఎవరిది – job యజమాని దా, server ది యా?	"రోజూ ఉదయం 9" అంటే ఎవరి ఉదయం? DST lo ఒక రోజు 23 గంటలే ఉంటుంది
ఒక job ఇంకొక job పూర్తయ్యాక నడవాలా?	అవును అంటే ఇది scheduler కాదు, workflow engine . పరిధి పెద్దది

ఒక ప్రశ్న అడగకూడదు - మరియు దాని బదులు ఏమి చెప్పాలి

"Exactly-once delivery కావాలా?" అని అడగవద్దు. అడిగితే, ఆ లక్షణం **సాధ్యమని మీరు అనుకుంటున్నారని** అర్థం.

బదులుగా ఇలా చెప్పండి: "Exactly-once **execution** సాధ్యం కాదు – ఆగిపోయిన worker ని ఆపడం మన చేతిలో లేదు. కానీ exactly-once **effect** సాధ్యం: రెండు workers నడిచినా, **ఒక్క ఫలితమే** అంగీకరించబడేలా చేయగలం."

ఈ ఒక్క వాక్యం మిమ్మల్ని మిగతా అభ్యర్థుల నుంచి వేరు చేస్తుంది, మరియు §8 మొత్తం దీనికి రుజువు.

Part 2 – మొదటి విరుపు: అందరూ ఒకే క్షణంలో

3. Step – ఒక జాబితా, ఒక tick, ఒక retry

మొదట అందరూ రాసే code. ఇది తప్పు కాదు, ఇది అసంపూర్ణం.

```
class Scheduler {
  constructor() { this.jobs = []; }
  define(id, { every, run }) {
    this.jobs.push({ id, every, run, nextAt: 0 });
  }
  tick(now) {
    for (const job of this.jobs) {
      if (now < job.nextAt) continue;
      job.nextAt = now + job.every;
      this.attempt(job, 1);
    }
  }
  attempt(job, n) {
    job.run(err => {
      if (!err) return;
      setTimeout(() => this.attempt(job, n + 1), 1000); // విఫలమైతే ఆగి మళ్ళీ
    });
  }
}
```

ఇది పని చేస్తుంది. tick ప్రతి సెకనుకూ నడుస్తుంది, సమయం వచ్చిన jobs నడుస్తాయి, విఫలమైతే ఒక సెకను ఆగి మళ్ళీ ప్రయత్నిస్తాయి.

ఇప్పుడు ఒక విషయం గమనించండి – నిజమైన systems lo jobs ఎప్పుడు షెడ్యూల్ అవుతాయి?

అందరూ ఒకే సమయం రాస్తారు

మీ company lo 500 cron entries ఉంటే, వాటిలో ఎన్ని 0 * * * * (గంట మొదట్లో) ఉంటాయి? **దాదాపు అన్నీ.**

ఎందుకంటే "ప్రతి గంటకీ" అని అడిగినప్పుడు ఎవరూ "గంటకి 17వ నిమిషంలో" అని రాయరు. అందరూ **సున్నా** రాస్తారు.

అంటే మీ 500 jobs అన్నీ సరిగ్గా ఒకే మిల్లీసెకనులో మొదలవుతాయి.

దీన్ని కొలుద్దాం. 500 jobs, అన్నీ ఒకే క్షణం. అవి పిలిచే downstream service ఒకేసారి **60 requests** భరిస్తుంది; అంతకు మించి వచ్చినవి తిరస్కరించబడతాయి (మరియు retry అవుతాయి).

4. మొదటి విరుపు – ఘాతీయ backoff ఏమి ఆదా చేయలేదు

మూడు విధానాలు పోల్చు: స్థిరంగా 1 సెకను, ఘాతీయ backoff (1, 2, 4, 8...), మరియు backoff + jitter.

నా ఊహ: "backoff స్పష్టంగా మెరుగు, jitter కొంచెం ఇంకా మెరుగు." **మొదటి భాగం పూర్తిగా తప్పు.**

500 jobs · అన్ని ఒకే క్షణంలో షెడ్యూల్ · downstream ఒకేసారి 60 భరిస్తుంది

విధానం	ప్రయత్నాలు	వృథా పిలుపులు	గరిష్ట ఉప్పెన	అన్ని అయిన క్షణం
స్థిరంగా 1 s	2340	1840	500	8.2 s
ఘాతీయ backoff	2340	1840	500	183.2 s
backoff + jitter	1208	708	747	4.8 s

మొదటి రెండు వరుసలు – సంఖ్యలు సరిగ్గా ఒకటే

స్థిరంగా 1 సెకను: **2,340** ప్రయత్నాలు, **1,840** వృథా.

ఘాతీయ backoff: **2,340** ప్రయత్నాలు, **1,840** వృథా.

ఒక్క పిలుపు కూడా ఆదా కాలేదు. కానీ పూర్తవడానికి 8.2 సెకన్లకు బదులు **183.2 సెకన్లు – 22 రెట్లు ఎక్కువ.**

అంటే ఈ పరిస్థితిలో ఘాతీయ backoff **స్పష్టంగా నష్టం.** అదే వృథా, అదే ఉప్పెన, కేవలం చాలా నెమ్మది.

ఎందుకు ఇలా జరిగింది? 500 jobs **ఒకే క్షణంలో** విఫలమయ్యాయి. అందరూ ఒకే సూత్రంతో backoff లెక్కిస్తారు. కాబట్టి అందరూ **ఒకే క్షణంలో** మళ్ళీ ప్రయత్నిస్తారు.

Backoff ఎదురుచూపు సమయాన్ని పెంచుతుంది. అది ఎదురుచూసేవాళ్ళని విడదీయదు.

ప్రతి 500 ms కిటికీలో ఎన్ని requests వచ్చాయో చూస్తే ఇది స్పష్టంగా కనిపిస్తుంది:

500 ms కిటికీకి వచ్చిన requests (మొదటి 10 సెకన్లు):

స్థిరంగా	500	0 440	0 380	0 320	0 260	0 200	0 140	0 80	0 20	0							
0 0																	
backoff	500	0 440	0 0	0 380	0 0	0 0	0 0	0 0	0 320	0 0							
0 0 0																	
+ jitter	747	260 81	54 42	13 1	3 2	5 0	0 0	0 0	0 0	0 0							
0 0 0																	
సమయం (s)	0.0	0.5	1.0	1.5	2.0	2.5	3.0	3.5	4.0	4.5	5.0	5.5	6.0	6.5	7.0	7.5	8.0
8.5	9.0	9.5															

మొదటి రెండు వరుసల్లో సంఖ్యలు **సరిగ్గా ఒకటే** – 500, 440, 380, 320. కేవలం వాటి మధ్య ఖాళీలు పెరిగాయి.

ఒకే ఎత్తు, వేరే అంతరం - backoff ఉప్పునని తగ్గించలేదు, దూరం చేసింది

స్థిరంగా 1 s

500

440

380

320

260

ఘాతీయ backoff

500

440

380

320

backoff + jitter

747

260

...

...

...

...

...

...

...

...

...

...

...

...

...

...

...

...

...

...

...

...

...

...

...

...

...

...

...

...

...

...

...

...

...

...

...

...

...

...

...

...

...

...

...

...

...

...

...

...

...

...

Backoff ఎదురుచూపుని పెంచుతుంది. అది ఎదురుచూసేవాళ్ళని విడదీయదు.

అందరూ ఒకే క్షణంలో విఫలమైతే, అందరూ ఒకే క్షణంలో తిరిగి వస్తారు -

1 సెకను తర్వాత అయినా, 64 సెకన్ల తర్వాత అయినా.

5. Step – jitter · మరియు షెడ్యూల్‌నే పరచడం

Jitter అంటే: ఎదురుచూపు సమయాన్ని **యాదృచ్ఛికం** చేయడం. 4 సెకన్లు కాదు - 0 నుంచి 4 సెకన్ల మధ్య ఏదో ఒకటి.

```
const Backoff = {
  fixed: ms => () => ms,
  exp: (base, cap) => n => Math.min(cap, base * 2 ** (n - 1)),
  // "full jitter" - 0 నుంచి పరిమితి వరకు ఏదైనా
  jitter: (base, cap) => (n, rnd) =>
    Math.floor(rnd() * Math.min(cap, base * 2 ** (n - 1))),
};
```

ఒక్క `rnd() *` - అంతే. దాని ఫలితం: వృథా పిలుపులు **1,840 → 708**, మరియు పూర్తవడం **8.2 s → 4.8 s**.

కానీ పట్టికలో ఇంకొక సంఖ్య ఉంది, మరియు అది ఇబ్బందికరమైనది: jitter యొక్క **గరిష్ట ఉప్పున 747** - స్థిరమైన దాని 500 కంటే **ఎక్కువ**.

ఆ 747 ఎక్కడి నుంచి వచ్చింది - మరియు అది ఏమి చెప్తోంది

మొదటి 500 ms కిటికీలో: 500 original + jitter వల్ల వెంటనే తిరిగి వచ్చిన 247 = **747**.

అంటే jitter **retry ఉప్పునని** పరిచింది, కానీ **మొదటి ఉప్పునని** ముట్టుకోలేదు. ఎందుకంటే ఆ 500 jobs మొదలవడం retry కాదు - అది **షెడ్యూల్**.

కాబట్టి అసలు పరిష్కారం ఇంకా వెనక్కి ఉంది: **షెడ్యూల్‌నే పరచాలి**.

Job ని నిర్వచించేటప్పుడు ఒక `jitterMs` ఇద్దాం - "ఈ job ని ఈ కిటికీలో ఎక్కడైనా మొదలుపెట్టు":

```
#create(job, scheduledFor) {
  const jitter = job.jitterMs ? Math.floor(this.#rnd() * job.jitterMs) : 0;
  const run = { jobId: job.id, scheduledFor, runAt: this.#sim.now + jitter, ... };
}
```

```
this.#sim.after(jitter, () => this.#ready.push(run));
}
```

ముఖ్యమైన వివరం: `scheduledFor` మారలేదు – అది ఇంకా సరిగ్గా గంట మొదలే. మారింది `runAt` మాత్రమే. ఇది §12 lo చాలా ముఖ్యమవుతుంది.

ఇప్పుడు నాలుగో వరుసతో కలిపి చూద్దాం:

విధానం	ప్రయత్నాలు	వృథా పిలుపులు	గరిష్ట ఉప్పెన	అన్నీ అయిన క్షణం
స్థిరంగా 1 s	2340	1840	500	8.2 s
ఘాతీయ backoff	2340	1840	500	183.2 s
backoff + jitter	1208	708	747	4.8 s
+ షెడ్యూల్ కూడా 60 s lo పరచడం	500	0	13	60.1 s

చివరి వరుస – 500 JOBS కి సరిగ్గా 500 ప్రయత్నాలు

వృథా పిలుపులు: 0. గరిష్ట ఉప్పెన: 13 (500 కి బదులు).

Downstream ఎప్పుడూ ఇబ్బంది పడలేదు, కాబట్టి ఏదీ విఫలం కాలేదు, కాబట్టి retry అవసరమే రాలేదు. **మేము retry ని మెరుగుపరచలేదు – retry అవసరాన్నే తొలగించాం.**

ఖరీదు: అన్నీ పూర్తవడానికి **60.1 సెకన్లు** (4.8 కి బదులు). ఎందుకంటే మనమే వాటిని 60 సెకన్లలో పరిచాం. అంటే:

ఇది ఒక ఒప్పందం, ఉచిత లాభం కాదు

"గంట మొదట్లో ఖచ్చితంగా" అనేది **12:00:08 కి అన్నీ అవుతాయి** నుంచి **12:01:00 కి అన్నీ అవుతాయి** గా మారుతుంది.

చాలా jobs కి ఇది పట్టింపు లేదు – "గంటకోసారి cache refresh" కి 60 సెకన్ల తేడా ఏమీ కాదు. కొన్నిటికి పట్టింపు ఉంటుంది.

అందుకే `jitterMs` **ప్రతి job కీ వేరుగా** ఉంటుంది, అందరికీ ఒకటే కాదు. ఇది §11 lo మళ్ళీ వచ్చే ఆలోచన.

మొదటి విరుపు పూర్తయింది. ఇప్పుడు ఒక job నడుస్తోంది. కానీ **ఎవరు** నడుపుతున్నారు?

Part 3 – రెండో విరుపు: ఈ పని ఎవరిది?

6. Step – lease • "ఇది నాది, ఇంత సమయం వరకు"

ఒక worker ఒక job ని తీసుకున్నాక అది పని చేస్తుంది. కానీ ఆ worker **మాయమైతే**? Machine చనిపోవచ్చు, process చావొచ్చు, network తెగిపోవచ్చు.

ఆ పని **ఎప్పటికీ** జరగకూడదా? కాదు – ఇంకొకరు దాన్ని తీసుకోవాలి. కానీ ఎప్పుడు?

సాధారణ సమాధానం **lease**: "ఈ పని నీది, కానీ **30 సెకన్ల వరకే**. ఆలోపు 'అయిపోయింది' అని చెప్పకపోతే, నేను దాన్ని ఇంకొకరికి ఇచ్చేస్తాను."

```
claim() {
  const run = this.#ready.shift();
  if (!run) return null;
  run.state = 'running';
  run.claimedAt = this.#sim.now;      // ← ఇక్కడి నుంచి lease మొదలు
  return run;
}

#reapExpired() {
  for (const run of this.#runs.values()) {
    if (run.state !== 'running') continue;
    if (this.#sim.now - run.claimedAt <= this.leaseMs) continue;
    this.#reschedule(run, 'LEASE_EXPIRED'); // ఇంకొకరికి ఇచ్చేయ్
  }
}
```

ఇది సరళంగా ఉంది. కానీ **lease ఎంత ఉండాలి?**

అదే ప్రశ్న ఈ మొత్తం Part కి మూలం, ఎందుకంటే దానికి రెండు వైపులా నష్టం ఉంది.

7. రెండో విరుపు – 16.4% jobs ఒకేసారి రెండుసార్లు

కొలుద్దాం. 3,000 jobs. Job వ్యవధులు **ఒకేలా ఉండవు** – నిజమైన systems lo ఎప్పుడూ ఒక **బరువైన తోక** ఉంటుంది:

- 80% jobs – 1 సెకను లోపు
- 17% – 1 నుంచి 5 సెకన్లు
- 3% – **5 నుంచి 30 సెకన్లు**

మరియు 2% workers పని మధ్యలో చచ్చిపోతారు.

3,000 jobs · 2% workers పని మధ్యలో చచ్చిపోతారు · job వ్యవధి బరువైన తోక
(80% jobs 1 s లోపు · 17% 1–5 s · 3% 5–30 s)

lease	heartbeat	ఏకకాలంలో రెండుసార్లు	శాతం	చచ్చిన job ఆగిన సమయం (మధ్య / గరిష్ఠ)
2 s లేదు		491	16.4%	1.7 s / 2.0 s
5 s లేదు		86	2.9%	4.7 s / 5.0 s
10 s లేదు		78	2.6%	9.7 s / 10.0 s
30 s లేదు		0	0.0%	29.7 s / 30.0 s
60 s లేదు		0	0.0%	59.7 s / 60.0 s

ఈ పట్టికలో రెండు నిలువు వరుసలు వ్యతిరేక దిశల్లో కదులుతున్నాయి

2 సెకన్ల lease: చచ్చిన job 1.7 సెకన్లలో తిరిగి మొదలవుతుంది – అద్భుతం. కానీ **491 jobs (16.4%) ఒకేసారి ఇద్దరు workers మీద నడిచాయి.**

60 సెకన్ల lease: ఏ job ఒకేసారి రెండుసార్లు నడవలేదు – అద్భుతం. కానీ ఒక worker చచ్చే ఆ పని **59.7 సెకన్లు ఆగిపోతుంది.**

Lease ని ఎటు కదిపినా ఒక వైపు బాగుపడి ఇంకో వైపు చెడిపోతుంది. **మధ్యలో మంచి విలువ లేదు.**

ఆ 16.4% ఎక్కడి నుంచి వచ్చాయి? **బరువైన తోక నుంచి.** 3% jobs 5–30 సెకన్లు పడతాయి. Lease 2 సెకన్లు అయితే, అవి **ప్రతిసారీ** గడువు దాటతాయి – పని ఇంకా సవ్యంగా నడుస్తున్నా.

ఇది ఎందుకు ఇంత సాధారణమైన తప్పు

Lease ని ఎంచుకునేటప్పుడు అందరూ **సగటు** చూస్తారు. ఇక్కడ సగటు వ్యవధి ఒక సెకను కంటే తక్కువ – కాబట్టి "2 సెకన్లు ధారాళం" అనిపిస్తుంది.

కానీ lease ని నిర్ణయించేది సగటు కాదు, **తోక**. మరియు తోక ఎప్పుడూ మీరు అనుకున్నదాని కంటే పొడవుగా ఉంటుంది.

8. Step – heartbeat · మరియు ఆగిపోయిన worker

సమస్య మూలం ఇది: lease **పని మొదలైనప్పుడు** లెక్క వేస్తోంది. కానీ మనం తెలుసుకోవాలనుకుంటున్నది "పని ఎంతసేపు పడుతోంది" కాదు – **"ఈ worker ఇంకా బతికున్నాడా"**.

ఆ రెండూ వేర్వేరు ప్రశ్నలు. వాటిని విడదీద్దాం: worker ప్రతి సెకనుకూ **"నేను ఇంకా ఉన్నాను"** అని చెప్తాడు.

```
heartbeat(key, fence) {
  const run = this.#runs.get(key);
  if (!run || run.fence !== fence || run.state !== 'running') return false;
  run.lastBeat = this.#sim.now;    // ← lease ఇక్కడి నుంచి మళ్ళీ మొదలు
  return true;
}

#reapExpired() {
```

```

for (const run of this.#runs.values()) {
  if (run.state !== 'running') continue;
  // "మొదలైనప్పటి నుంచి" కాదు - "చివరి కబురు నుంచి"
  if (this.#sim.now - run.lastBeat <= this.heartbeatMs + this.leaseMs) continue;
  this.#stats.expired++;
  this.#reschedule(run, 'LEASE_EXPIRED');
}
}

```

ఇప్పుడు పని 30 సెకన్లు పట్టినా ఫరవాలేదు – worker ప్రతి సెకనుకూ కబురు పంపుతున్నాడు. కానీ అతను **చస్తే** కబురు ఆగుతుంది, మరియు 3 సెకన్లలో ఇంకొకరు తీసుకుంటారు:

2 s		1 s కొకటి		0		0.0%		3.0 s / 3.0 s
5 s		1 s కొకటి		0		0.0%		6.0 s / 6.0 s

రెండు వైపులా గెలిచాం

60 సెకన్ల lease యొక్క **భద్రత** (0% duplicates) మరియు 2 సెకన్ల lease యొక్క **వేగం** (3 సెకన్లలో కోలుకోవడం) – రెండూ ఒకేసారి.

ఎందుకంటే మనం ఇప్పుడు **సరైన ప్రశ్న** అడుగుతున్నాం: "పని ఎంతసేపు పడుతోంది" కాదు, "**worker బతికున్నాడా**".

కానీ – heartbeat ఒక రంధ్రాన్ని మూయలేదు

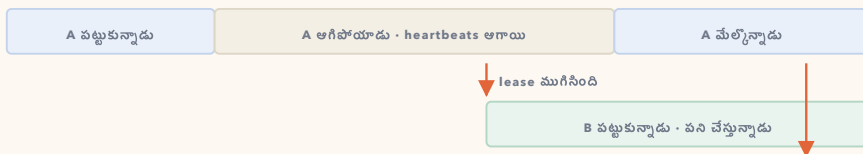
Worker **చావలేదు**. అతను **ఆగిపోయాడు**.

GC pause. VM migration. CPU ఆకలి. Laptop మూత. ఈ అన్నిటిలోనూ process సజీవంగానే ఉంది – అది కేవలం **కొన్ని సెకన్ల పాటు ఏమీ చేయదు**. Heartbeats ఆగుతాయి. Lease ముగుస్తుంది. ఇంకో worker మొదలుపెడతాడు.

తర్వాత మొదటి worker **మేల్కొంటాడు**. అతనికి తాను ఆగిపోయానని **తెలియదు**. అతను తన పని పూర్తి చేసి, తన ఫలితాన్ని రాస్తాడు.

ఆగిపోయిన worker – అతను చావలేదు, కాబట్టి ఎవరూ అతన్ని ఆపలేరు

సమయం



A మేల్కొని తన ఫలితాన్ని రాస్తాడు – అది **B ఫలితాన్ని తుడిచేస్తుంది**.

A కి తాను ఆగిపోయానని తెలియదు. అతని దృష్టిలో అంతా మామూలే.

Heartbeat దీన్ని ఆపలేదు – heartbeat ఆగిపోయిన వాడిని **గుర్తిస్తుంది, ఆపదు**.

4,000 jobs lo ఇది **106 సార్లు** జరిగింది – 2.65%.

కొలిచి చూద్దాం. 4,000 jobs, 3% workers lease దాటేంతసేపు ఆగిపోతారు. చివరికి **నిల్వలో ఎవరి ఫలితం ఉంది** అని పరీక్షిస్తాం:

4,000 jobs · 3% workers lease దాటంతనేపు ఆగిపోతారు (GC pause, VM migration)
heartbeat 1000 ms · lease 2000 ms

విధానం	writes	తిరస్కరించినవి	తుదికి పాత ఫలితం ఉన్న jobs
fencing లేకుండా	4111	0	106 (2.65%)
fencing token తో	4004	107	0 (0.00%)

పరిష్కారం: ప్రతి claim కి ఒక పెరుగుతున్న సంఖ్య ఇవ్వడం, మరియు రాసేటప్పుడు ఆ సంఖ్యని అడగడం.

```
claim() {
  ...
  run.fence = ++this.#fence;          // ఈ ప్రయత్నానికి మాత్రమే · ఎప్పుడూ పెరుగుతుంది
  return { key: run.key, ..., fence: run.fence };
}

complete(key, fence, result) {
  const run = this.#runs.get(key);
  if (!run) return false;
  // పాత worker · లేదా ఇప్పటికే పూర్తయింది
  if (run.fence !== fence) { this.#stats.fenced++; return false; }
  if (run.state !== 'running') { this.#stats.fenced++; return false; }
  run.state = 'done'; run.result = result;
  return true;
}
```

DD 24 §8 lo చూసిన token ఇదే – కానీ అక్కడ అది ఒక **timer** ని ఆపింది; ఇక్కడ అది ఒక **worker** ని ఆపుతోంది.

దీన్ని నిజంగా పరీక్షించాలంటే ఆ ఒక్క క్రమాన్నే నడిపి చూడాలి:

```
==== మార్పు లేని code ====
A పట్టుకున్నాడు · fence 1
B పట్టుకున్నాడు · fence 2
A రాయడానికి ప్రయత్నించాడు → తిరస్కరించబడింది
B రాశాడు → అంగీకరించబడింది
తుది ఫలితం: B ఫలితం · state done

==== fence పరీక్ష తీసేస్తే ====
A పట్టుకున్నాడు · fence 1
B పట్టుకున్నాడు · fence 2
A రాయడానికి ప్రయత్నించాడు → ▲ అంగీకరించబడింది
B రాశాడు → ▲ తిరస్కరించబడింది
తుది ఫలితం: A ఫలితం · state done
```

రెండే భాగంలో రెండు తప్పులు జరిగాయి, ఒకటి కాదు

ఒకటి: ఆగిపోయిన A యొక్క పాత ఫలితం అంగీకరించబడింది.

రెండు: ఆ తర్వాత సరైన worker B తిరస్కరించబడ్డాడు – ఎందుకంటే run అప్పటికే 'done'.

అంటే తప్పు ఫలితం నిల్వ అయింది, మరియు సరైనది ఎప్పటికీ రాయబడలేదు. మరియు ఎక్కడా ఒక్క error కూడా లేదు.

ఈ PART నుంచి అసలు పాఠం – మరియు ఇది INTERVIEW LO చెప్పాల్సిన వాక్యం

ఒక job రెండుసార్లు నడవకుండా ఆపడం అసాధ్యం. రెండే ఫలితం అంగీకరించబడకుండా ఆపడం సులభం.

ఆగిపోయిన worker మన అదుపులో లేడు. అతన్ని చంపలేం, ఆపలేం, హెచ్చరించలేం. కానీ అతను తిరిగి వచ్చి **రాయబడేటప్పుడు** – ఆ క్షణంలో అతను మన తలుపు తడతాడు. అక్కడ మనం "నువ్వు పాతవాడివి" అని చెప్పగలం.

అందుకే ఇది **exactly-once execution** కాదు, **exactly-once effect**.

Part 4 – మూడో విరుపు: తప్పిపోయిన runs

9. Step – scheduler 6 గంటలు పడిపోయింది

ఇప్పటివరకు workers విఫలమవడం చూశాం. ఇప్పుడు **scheduler స్వయంగా** పడిపోతే?

200 jobs, ప్రతిదీ 5 నిమిషాలకు ఒకసారి. Scheduler రాత్రి 2 నుంచి ఉదయం 8 వరకు పడిపోయింది – **6 గంటలు**.

అది తిరిగి లేచినప్పుడు **14,400 runs తప్పిపోయి ఉన్నాయి** (200×72).

మొదటి భయం: "ఇవన్నీ ఒకసారి నడిస్తే system మునిగిపోతుంది." దాన్ని కొలుద్దాం:

200 jobs · ప్రతిదీ 5 నిమిషాలకు ఒకసారి · scheduler 6 గంటలు పడిపోయింది
తప్పిపోయిన runs: $200 \times 72 = 14,400$

విధానం	queue lo చేరినవి	నడిచినవి	వదిలేసినవి	queue ఖాళీ అయిన సమయం
అన్నీ నడుపు	14,400	14,400	0	12 నిమిషాలు
అన్నీ వదిలేయ్	0	0	14,400	0 నిమిషాలు
చివరిది ఒకటే	200	200	14,200	1 నిమిషాలు
చివరి 30 నిమిషాలవి	1,200	1,200	13,200	1 నిమిషాలు

నా భయం తప్పు – భారం సమస్య కాదు

14,400 runs **12 నిమిషాల్లో** పూర్తయ్యాయి. అది ఇబ్బందికరం కావచ్చు, కానీ అది ఒక విపత్తు కాదు.

నేను ఈ విరుపుని "catch-up storm system ని కూల్చేస్తుంది" అని రాయబోయాను. కొలిచాక అది నిజం కాదని తేలింది – కాబట్టి ఆ కథని వదిలేశాను.

నిజమైన సమస్య భారం కాదు. అది సరి-తప్పు.

10. మూడో విరుపు – సరైన ఒక్క విధానం అంటూ లేదు

ఆ 14,400 runs అన్నీ ఒకేలాంటివి కావు. ఒక నిజమైన fleet ఇలా ఉంటుంది:

job రకం	అన్నీ నడిపితే ఏమవుతుంది	వదిలేస్తే ఏమవుతుంది
ప్రతి user కి రోజువారీ email	72 emails ఒక్కొక్కరికి	ఒక email ఆలస్యం
Subscription బిల్లింగ్ (period కొకటి)	సరైనదే	72 periods బిల్లు కాలేదు

job రకం	అన్నీ నడిపితే ఏమవుతుంది	వదిలేస్తే ఏమవుతుంది
Cache refresh	72 సార్లు వృథా పని	ఏమీ కాదు
Health check	72 సార్లు వృథా పని	ఏమీ కాదు

ఇప్పుడు ప్రతి **ఒక విధానాన్ని** అందరికీ వర్తింపజేస్తే ఏమవుతుందో కొలుద్దాం:

Scheduler 6 గంటలు పడిపోయింది • ప్రతి job 5 నిమిషాలకు ఒకసారి → 72 runs తప్పాయి

job రకం		సరైన విధానం అన్నీ		వదిలేయ్ ఒకటే		per-job	
ప్రతి user కి రోజువారీ email		one	2880	0	40	40	
subscription బిల్లింగ్ (period కొకటి)		all	1440	0	20	1440	
cache refresh		one	5760	0	80	80	
health check		skip	4320	0	60	0	
విధానం		అదనంగా నడిచినవి (తప్పుడు పని)		జరగని runs (పోయిన పని)			
అన్నీ నడుపు		12,840		0			
అన్నీ వదిలేయ్		0		1,560			
చివరిది ఒకటే		60		1,420			
per-job		0		0			

మూడు "సహజమైన" విధానాలు - మూడూ తప్పు

అన్నీ నడుపు: 12,840 తప్పుడు runs. అందులో **2,880 అదనపు emails** - మీ users కి ఒక్కొక్కరికీ 72 emails. Support team కి ఇది ఒక సంక్షోభం.

అన్నీ వదిలేయ్: 1,560 runs పోయాయి. అందులో **1,440 బిల్లింగ్ periods** - అంటే నిజమైన డబ్బు. ఎవరూ దీన్ని గమనించరు, ఎందుకంటే ఏ error రాదు.

చివరిది ఒకటే: ఇది "సమతుల్యం" లా అనిపిస్తుంది. కానీ ఇది ఇంకా **1,420 runs** కోల్పోతుంది - బిల్లింగ్ తో సహా.

per-job: తప్పుడు పని 0, పోయిన పని 0.

ఇదే ఈ **PART** యొక్క పాఠం, మరియు ఇది ఒక **DESIGN** పాఠం

"తప్పిపోయిన runs ని ఏమి చెయ్యాలి" అనేది ఒక engineering నిర్ణయం కాదు. అది ఒక business నిర్ణయం.

Scheduler కి ఆ సమాధానం **తెలియదు**, మరియు తెలియకూడదు. బిల్లింగ్ ప్రతి period కీ నడవాలో లేదో scheduler రచయితకి ఎలా తెలుస్తుంది?

కాబట్టి scheduler పని ఆ నిర్ణయం తీసుకోవడం కాదు - ఆ నిర్ణయాన్ని job యజమాని చెప్పగలిగేలా చేయడం.

11. Step – విధానాన్ని job యజమాని చెప్తాడు

కాబట్టి catch-up విధానాన్ని ఒక విలువగా చేద్దాం, `if` పరంపరగా కాదు:

```
const CatchUp = {
  all: missed => missed, // ప్రతి periodకి ఒకటి (బిల్లింగ్)
  skip: () => [], // ఏదీ వద్దు (health check)
  latest: missed => missed.slice(-1), // చివరిది ఒకటి (cache, report)
  within: ms => (missed, now) => missed.filter(t => now - t <= ms),
};
```

ఇప్పుడు job నిర్వచనం తన సమాధానాన్ని తానే మోసుకొస్తుంది:

```
sched.define('subscription-billing', { every: HOUR, catchUp: CatchUp.all });
sched.define('daily-digest', { every: DAY, catchUp: CatchUp.latest });
sched.define('health-check', { every: MIN, catchUp: CatchUp.skip });
sched.define('rebuild-index', { every: HOUR, catchUp: CatchUp.within(2*HOUR)
});
```

మరియు `tick` lo ఒక్క పంక్తి:

```
tick() {
  const now = this.#sim.now;
  for (const job of this.#jobs.values()) {
    const missed = [];
    for (let t = job.lastScheduled + job.every; t <= now; t += job.every)
      missed.push(t);
    if (!missed.length) continue;
    job.lastScheduled = missed[missed.length - 1];
    const chosen = job.catchUp(missed, now); // ← job తన విధానాన్ని తానే చెప్తుంది
    this.#stats.caughtUp += chosen.length;
    this.#stats.skipped += missed.length - chosen.length;
    for (const at of chosen) this.#create(job, at);
  }
}
```

`tick` కి `all`, `skip`, `latest` అంటే ఏమిటో తెలియదు. అది కేవలం ఒక function పిలుస్తుంది. కొత్త విధానం కావాలంటే scheduler ని ముట్టుకోనవసరం లేదు.

ఇక్కడ ఒక సూక్ష్మమైన విషయం ఉంది – MISSED ఎప్పుడూ అన్నీ లెక్కిస్తుంది

`catchUp` ని పిలిచే ముందు మనం తప్పిపోయిన అన్ని క్షణాలనూ లెక్కిస్తాం, తర్వాత job వాటిలో కొన్నిటిని ఎంచుకుంటుంది.

దీని వల్ల `skipped` అనే లెక్క నిజాయితీగా ఉంటుంది – "72 తప్పాయి, 1 నడిపాం, 71 వదిలేశాం". **ఎంత వదిలేశామో మనకు తెలుసు**, మరియు అది dashboard lo కనిపిస్తుంది.

`catchUp` లోపలే లెక్కపెడితే ఈ సంఖ్య పోతుంది, మరియు "మేము ఎంత పని వదిలేశాం" అనే ప్రశ్నకి ఎప్పటికీ సమాధానం ఉండదు.

ఢూడే విరుపు పూర్తయింది. ఇప్పుడు అన్నిటినీ కలిపే ఒక్క ఆలోచన మిగిలింది.

Part 5 – పూర్తి system

12. Step – (job, ఏ క్షణానికి) – ఒకే ఒక్క key

మూడు విరుపుల్లోనూ ఒకే ప్రశ్న పదేపదే వచ్చింది, వేరే వేషాల్లో:

- §5 lo: jitter వల్ల job అలస్యంగా నడిస్తే, అది ఏ run?
- §8 lo: రెండు workers నడిపితే, అది ఒకే run అని ఎలా చెప్పాలి?
- §11 lo: scheduler రెండుసార్లు tick చేస్తే, అది అదే run నా, కొత్తదా?

ఈ మూడింటికీ ఒకే సమాధానం: **ఒక run ని దాని (jobId, scheduledFor) జత గుర్తిస్తుంది.**

```
#create(job, scheduledFor) {
  const key = `${job.id}@${scheduledFor}`;
  if (this.#runs.has(key)) { this.#stats.deduped++; return; } // ఇది ఇప్పటికే ఉంది
  const jitter = job.jitterMs ? Math.floor(this.#rnd() * job.jitterMs) : 0;
  const run = { key, jobId: job.id, scheduledFor, runAt: this.#sim.now + jitter,
    attempt: 0, state: 'pending', fence: 0, lastBeat: 0 };
  this.#runs.set(key, run);
  ...
}
```

ఇక్కడ **scheduledFor ఉంది, runAt కాదు** – ఇది ముఖ్యం.

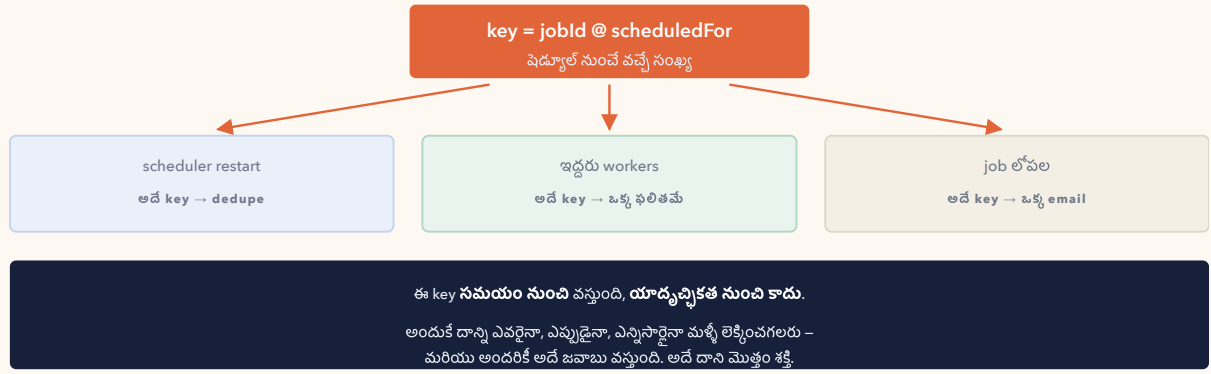
ఎందుకు SCHEDULED_FOR, DATE_NOW() కాదు?

ఎందుకంటే **scheduledFor** అనేది షెడ్యూల్ నుంచి వచ్చే సంఖ్య – అదే job, అదే నియమం, అదే క్షణం నుంచి ఎప్పుడూ అదే విలువ వస్తుంది.

కాబట్టి scheduler పడిపోయి, తిరిగి లేచి, ఒక పాత checkpoint నుంచి కొనసాగినా – అది అదే key ని లెక్కిస్తుంది, మరియు **#runs.has(key)** "ఇది ఇప్పటికే ఉంది" అని చెప్పుంది.

Date.now() వాడితే ప్రతి restart కొత్త key తయారు చేస్తుంది, మరియు **ప్రతి restart కి jobs మళ్ళీ నడుస్తాయి.**

మరియు అదే key ఒక **idempotency key** గా job కి కూడా ఇవ్వొచ్చు. Job ఒక email పంపుతుంటే, అది **sendEmail(user, key)** అని పిలిస్తే – email service అదే key ని రెండేసారి చూసి **పంపకుండా** ఉంటుంది. అలా exactly-once effect **job లోపల కూడా** కొనసాగుతుంది.



13. మొత్తం code · 2,000 ప్రయోగాలు · mutation testing

ఇప్పుడు అన్నీ ఒక చోట.

```

const CatchUp = {
  all: missed => missed,
  skip: () => [],
  latest: missed => missed.slice(-1),
  within: ms => (missed, now) => missed.filter(t => now - t <= ms),
};

const Backoff = {
  fixed: ms => () => ms,
  exp: (base, cap) => n => Math.min(cap, base * 2 ** (n - 1)),
  jitter: (base, cap) => (n, rnd) =>
    Math.floor(rnd() * Math.min(cap, base * 2 ** (n - 1))),
};

class Scheduler {
  #sim; #rnd; #jobs = new Map(); #runs = new Map(); #ready = [];
  #fence = 0;
  #stats = { created: 0, claimed: 0, completed: 0, failed: 0, retried: 0,
    dead: 0, expired: 0, fenced: 0, deduped: 0, caughtUp: 0, skipped: 0 };

  constructor(sim, { rnd = Math.random, leaseMs = 30_000,
    heartbeatMs = 10_000 } = {}) {
    this.#sim = sim; this.#rnd = rnd;
    this.leaseMs = leaseMs; this.heartbeatMs = heartbeatMs;
  }

  define(id, { every, jitterMs = 0, catchUp = CatchUp.latest,
    maxAttempts = 5, backoff = Backoff.jitter(1000, 60_000) }) {
    this.#jobs.set(id, { id, every, jitterMs, catchUp, maxAttempts, backoff,
      lastScheduled: this.#sim.now });
    return this;
  }
}
  
```

\$11 – tick. తప్పిపోయినవి లెక్కించి, job ని ఏమి చెయ్యాలో అడుగుతుంది:

```

tick() {
  const now = this.#sim.now;
  for (const job of this.#jobs.values()) {
    const missed = [];
    for (let t = job.lastScheduled + job.every; t <= now; t += job.every)
      missed.push(t);
    if (!missed.length) continue;
    job.lastScheduled = missed[missed.length - 1];
    const chosen = job.catchUp(missed, now);
    this.#stats.caughtUp += chosen.length;
    this.#stats.skipped += missed.length - chosen.length;
    for (const at of chosen) this.#create(job, at);
  }
}

```

§12 – ఒకే key, మరియు §5 యొక్క jitter:

```

#create(job, scheduledFor) {
  const key = `${job.id}@${scheduledFor}`;
  if (this.#runs.has(key)) { this.#stats.deduped++; return; }
  const jitter = job.jitterMs ? Math.floor(this.#rnd() * job.jitterMs) : 0;
  const run = { key, jobId: job.id, scheduledFor,
    runAt: this.#sim.now + jitter,
    attempt: 0, state: 'pending', fence: 0, lastBeat: 0 };
  this.#runs.set(key, run);
  this.#stats.created++;
  this.#sim.after(jitter, () => {
    if (run.state === 'pending') this.#ready.push(run);
  });
}

```

§8 – claim ఒక fence ఇస్తుంది:

```

claim() {
  this.#reapExpired();
  while (this.#ready.length) {
    const run = this.#ready.shift();
    if (run.state !== 'pending') continue;
    run.state = 'running';
    run.attempt++;
    run.fence = ++this.#fence; // ఈ ప్రయత్నానికి మాత్రమే
    run.lastBeat = this.#sim.now;
    this.#stats.claimed++;
    return { key: run.key, jobId: run.jobId, scheduledFor: run.scheduledFor,
      attempt: run.attempt, fence: run.fence };
  }
  return null;
}

heartbeat(key, fence) {
  const run = this.#runs.get(key);
  if (!run || run.fence !== fence || run.state !== 'running') return false;
  run.lastBeat = this.#sim.now;
  return true;
}

```

```

}

complete(key, fence, result) {
  const run = this.#runs.get(key);
  if (!run) return false;
  if (run.fence !== fence) { this.#stats.fenced++; return false; }
  if (run.state !== 'running') { this.#stats.fenced++; return false; }
  run.state = 'done'; run.result = result;
  this.#stats.completed++;
  return true;
}

fail(key, fence, error) {
  const run = this.#runs.get(key);
  if (!run || run.fence !== fence || run.state !== 'running') {
    this.#stats.fenced++; return false;
  }
  this.#stats.failed++;
  return this.#reschedule(run, error);
}

```

§5 – backoff, మరియు వదిలేయాల్సిన చోటు:

```

#reschedule(run, error) {
  const job = this.#jobs.get(run.jobId);
  if (run.attempt >= job.maxAttempts) {
    run.state = 'dead'; run.error = error; // dead letter – మౌనంగా పోకూడదు
    this.#stats.dead++;
    return false;
  }
  run.state = 'pending';
  this.#stats.retried++;
  this.#sim.after(job.backoff(run.attempt, this.#rnd), () => {
    if (run.state === 'pending') this.#ready.push(run);
  });
  return true;
}

#reapExpired() {
  for (const run of this.#runs.values()) {
    if (run.state !== 'running') continue;
    if (this.#sim.now - run.lastBeat <= this.heartbeatMs + this.leaseMs) continue;
    this.#stats.expired++;
    // fence ని ఇక్కడ రద్దు చెయ్యవసరం లేదు: తిరిగి పట్టుకున్నప్పుడు
    // ++this.#fence ఎప్పుడూ పెద్ద సంఖ్యే ఇస్తుంది, కాబట్టి పాత fence సరిపోదు.
    this.#reschedule(run, 'LEASE_EXPIRED');
  }
}

```

Scheduler పడిపోయి లేస్తే:

```

checkpoint() {
  return [...this.#jobs.values()].map(j => [j.id, j.lastScheduled]);
}

restore(cp) {

```

```

for (const [id, last] of cp) {
  const job = this.#jobs.get(id);
  if (job) job.lastScheduled = last;
}

get(key) { return this.#runs.get(key); }
runs() { return [...this.#runs.values()]; }

stats() {
  let pending = 0, running = 0, done = 0, dead = 0;
  for (const r of this.#runs.values())
    r.state === 'pending' ? pending++ : r.state === 'running' ? running++
    : r.state === 'done' ? done++ : dead++;
  return { ...this.#stats, pending, running, done, dead, total: this.#runs.size };
}
}

```

ఆరు నియమాలు

#	నియమం	విరిగితే అర్థం
1	<code>pending + running + done + dead === total</code>	ఒక run మాయమైంది
2	ఒక్కో (job, క్షణం) ఒక్కసారే పూర్తవ్వాలి	job రెండుసార్లు జరిగింది
3	<code>attempt</code> ఎప్పుడూ <code>maxAttempts</code> దాటకూడదు	retry ఆగడం లేదు
4	అంగీకరించిన ఫలితం ఇప్పటి worker దే కావాలి	పాత worker గెలిచాడు (§8)
5	అదే fence తో రెండేసారి <code>complete</code> → తిరస్కరించాలి	job ప్రభావం రెండుసార్లు
6	ఇద్దరు workers ఒకేసారి నడపడం ఒక ఉల్లంఘన కాదు	ఇది ఆశించినది – కింద చూడండి

నియమం 6 – నేను మొదట దీన్ని తప్పుగా రాశాను

నా మొదటి fuzz test "ఒక run ని ఇద్దరు workers ఒకేసారి నడపకూడదు" అని పరీక్షించింది. అది **2,000 to 1,848 ప్రయోగాల్లో విఫలమైంది**.

కొన్ని గంటలు code ని వెతికాక నాకు అర్థమైంది – **code తప్పు కాదు, నియమమే తప్పు**.

ఆగిపోయిన worker ని ఆపడం అసాధ్యం (§8). కాబట్టి "ఇద్దరు ఒకేసారి నడపకూడదు" అనేది సాధించలేని లక్ష్యం. సాధించగలిగినది – **ఒక్క ఫలితమే అంగీకరించబడాలి**, అదే నియమం 4 మరియు 5.

ఇది ఒక testing తప్పు కాదు, ఒక **అవగాహన** తప్పు. మరియు fuzz test దాన్ని బయటపెట్టింది.

2,000 యాదృచ్ఛిక ప్రయోగాలు · 2,10,118 runs తయారయ్యాయి

నియమ ఉల్లంఘనలు: 0

(ఏకకాలంలో ఇద్దరు workers నడిపినవి: 29,841 – ఇది ఆశించినది,

ఎందుకంటే ఆగిపోయిన worker ని ఆపడం అసాధ్యం. ముఖ్యమైనది – ఒక్క ఫలితమే అంగీకరించబడాలి.)

ఏ దారులు నడిచాయి:

created 2,10,118

claimed	2,87,641
completed	2,02,183
failed	34,464
retried	78,534
dead	6,279
expired	50,349
fenced	74,132
deduped	52,723
caughtUp	2,62,841
skipped	1,04,916

29,841 సార్లు ఇద్దరు workers ఒకే run ని ఒకేసారి నడిపారు – మరియు ప్రతిసారీ సరిగ్గా ఒక ఫలితం అంగీకరించబడింది. అదే మొత్తం design యొక్క రుజువు.

"0 ఉల్లంఘనలు" అంటే ఏమీ కాదు – test విఫలం కాగలదని ముందు నిరూపించాలి

ఇది DD 24 §13 lo చేసిన పనే, కానీ ఈసారి నాలుగు మార్పులతో. ప్రతిసారీ ఒక్క పంక్తి తీసేసి, fuzz పట్టుకుంటుందా అని చూశాను:

మార్పు లేని code : 0/400 విఫలం

```
fence పరీక్ష తీసేస్తే (complete lo) → 152/400 విఫలం
ఉదా: 89567ms: j1@80712 - రద్దైన worker (fence 204) ఫలితం గెలిచింది,
ఇప్పటిది fence 225
dedupe తీసేస్తే (#create lo) → 367/400 విఫలం
ఉదా: 10915ms: j0@8080 రెండుసార్లు పూర్తయింది
state పరీక్ష తీసేస్తే (complete lo) → 370/400 విఫలం
ఉదా: 3953ms: j3@2614 - అదే worker రెండేసారి రాయగలిగాడు
backoff తీసేసి వెంటనే మళ్ళీ ప్రయత్నిస్తే → 0/400 విఫలం
```

మొదటి మూడు పట్టుబడ్డాయి. **నాలుగోది పట్టుబడలేదు** – మరియు అది ఒక లోపం కాదు.

BACKOFF ఎందుకు పట్టుబడలేదు – మరియు అది ఏమి నేర్పుతుంది

Backoff తీసేసి వెంటనే retry చేస్తే, ఫలితాలు **ఇంకా సరైనవే**. ప్రతి job సరిగ్గా ఒకసారి పూర్తవుతుంది. ఏ నియమమూ విరగదు.

ఎందుకంటే **backoff ఒక సరి-తప్పు లక్షణం కాదు, అది ఒక భారపు లక్షణం**. దాని విలువ §4 పట్టికలో ఉంది (1,840 వ్యధా పీలుపులు), fuzz test lo కాదు.

రెండు రకాల లక్షణాలకి రెండు రకాల రుజువులు కావాలి:

- సరి-తప్పు లక్షణాలు → నియమాలు + fuzz
- భారం/వేగం లక్షణాలు → కొలతలు

ఒకదాన్ని ఇంకొకదానితో పరీక్షించడానికి ప్రయత్నిస్తే, ఒకటి తప్పుగా "సరే" అంటుంది.

మరియు ఒక దండగ పంక్తిని MUTATION TESTING బయటపెట్టింది

నేను మొదట `#reapExpired` lo `run.fence = 0;` అని రాశాను – "పాత worker ఇక రాయలేడు" అని.

దాన్ని తీసేసి పరీక్షిస్తే **0/400 విఫలం**. అంటే ఆ పంక్తి **ఏమీ చేయడం లేదు**.

కారణం: తిరిగి పట్టుకున్నప్పుడు `++this.#fence` ఎప్పుడూ **ఇంతకుముందు కంటే పెద్ద సంఖ్య** ఇస్తుంది. కాబట్టి పాత worker యొక్క fence ఎప్పటికీ సరిపోదు – మధ్యలో దాన్ని 0 చేసినా చేయకపోయినా.

కాబట్టి దాన్ని తీసేశాను. "జాగ్రత్త కోసం" అని ఉంచిన code, నిజానికి ఏమీ చేయనిది, చదివేవాడిని తప్పుదారి పట్టిస్తుంది – అది అవసరం అని అతను అనుకుంటాడు.

దశల నుంచి ఇక్కడికి – ఏమి చేరింది

ఎక్కడ	ఏమి జేడించాం	ఏమి బాగుపడింది
§3	<code>tick</code> , <code>retry</code>	<code>jobs</code> నడుస్తాయి
§5	<code>Backoff.jitter</code>	వృథా పిలుపులు 1,840 → 708
§5	<code>jitterMs</code> (షెడ్యూల్ పరచడం)	వృథా 708 → 0 , ఉప్పెన 500 → 13
§8	<code>heartbeat</code>	duplicates 16.4% → 0% , కోలుకోవడం 59.7 s → 3 s
§8	<code>fence</code>	పాత ఫలితం గెలవడం 2.65% → 0%
§11	<code>CatchUp</code> విలువలు	తప్పుడు పని 12,840 → 0 , పోయిన పని 1,560 → 0
§12	<code>jobId@scheduledFor</code>	restart కి 52,723 సార్లు dedupe
§13	<code>dead</code> స్థితి	విఫలమైనవి మౌనంగా పోవు

Part 6 – Interview Io

14. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి

నిమిషాలు	ఏమి చెప్పాలి
0-4	Queue కీ scheduler కీ తేడా. మూడు ప్రశ్నలు (ఎప్పుడు మళ్ళీ, ఎవరిది, తప్పిపోతే ఏమి) – ఇదే మీ ప్రణాళిక
4-8	§2 ప్రశ్నలు. "Exactly-once execution అసాధ్యం, exactly-once effect సాధ్యం" – ఇక్కడే చెప్పండి
8-13	<code>tick</code> + <code>retry</code> రాయండి. సులభం, త్వరగా దాటండి
13-21	మొదటి విరుపు. "అందరూ <code>0 * * * *</code> రాస్తారు" – <code>backoff</code> ఏమి అదా చేయదు అని చూపించండి. jitter, తర్వాత షెడ్యూల్ పరచడం
21-30	రెండో విరుపు. Lease యొక్క రెండు వైపుల నష్టం. heartbeat. తర్వాత "worker చావలేదు, ఆగిపోయాడు" → <code>fence</code>
30-37	మూడో విరుపు. నాలుగు job రకాలు board మీద రాసి, ఒకే విధానం అందరికీ ఎందుకు పని చేయడే చూపించండి
37-42	<code>(jobId, scheduledFor)</code> – మూడు సమస్యలూ ఒకే key తో ఎలా కలుస్తాయి
42-45	ఆరు నియమాలు, మరియు "నియమం 6 ని నేను మొదట తప్పుగా రాశాను"

ఏమి తప్పక చెప్పాలి

- Exactly-once execution అసాధ్యం; exactly-once effect సాధ్యం.** ఇది ఒక్క వాక్యం, మరియు ఇది మీ మొత్తం సమాధానాన్ని వేరే స్థాయిలో ఉంచుతుంది.
- Backoff ఎదురుచూపుని పెంచుతుంది, ఎదురుచూసేవాళ్ళని విడదీయదు.** Jitter లేని backoff synchronized clients కి దాదాపు వ్యర్థం.
- Lease "పని ఎంతసేపు" అని కొలవకూడదు, "worker బతికున్నాడా" అని కొలవాలి.**
- Worker చావడం కంటే ఆగిపోవడం ప్రమాదకరం** – చచ్చినవాడు తిరిగి రాడు, ఆగినవాడు తిరిగి వస్తాడు.
- Catch-up ఒక business నిర్ణయం.** Scheduler దాన్ని తీసుకోకూడదు, అడగాలి.

ఏమి వదిలేయాలి

- Cron వ్యక్తీకరణ (`* * * * *`) ని parse చేయడం – అది ఒక string parsing పని, design పని కాదు
- Heap vs sorted list – ఒక వాక్యంలో చెప్పి ముందుకి వెళ్ళండి
- Worker ఎలా jobs తెచ్చుకుంటాడు (poll vs push) – అడిగితే మాత్రమే
- DAG dependencies – అది వేరే problem (§16)

15. నీటితో చెప్పాల్సిన English script

"A queue delivers each item once. A scheduler delivers the *same* job over and over, on a promise you made in advance. That difference creates three questions a queue never has: when do I retry, who owns a running job, and what do I do about runs I missed while I was down. I'll build around those three."

"First, one thing I want to state up front: **exactly-once execution is not achievable**. A worker that's paused – GC, VM migration – isn't dead and isn't reachable, so I can't stop it. What is achievable is exactly-once **effect**: two workers may run, but only one result is ever accepted. That shapes everything else."

"Retry first. Here's the thing people miss: in a real system nearly every cron entry is `0 * * * *`, because nobody writes 'seventeen minutes past'. So all your jobs start on the same millisecond. I measured 500 synchronized jobs against a downstream that handles sixty at a time. Fixed one-second retry: 2,340 attempts, 1,840 of them wasted. Exponential backoff: **the same 2,340 and the same 1,840** – not one call saved – but it took 183 seconds instead of 8. **Backoff increases the wait; it doesn't separate the waiters**. Full jitter cuts the waste to 708. But the real fix is upstream: jitter the *schedule*, not just the retry. That takes it to 500 attempts and **zero waste**, at the cost of finishing by 12:01 instead of 12:00:08."

"Second, ownership. A worker takes a job and disappears – someone else has to pick it up, but when? A lease answers that, and choosing its length is a trap. With job durations that have a heavy tail – mine were 80% under a second but 3% up to thirty – a two-second lease made **16.4% of jobs run on two workers at once**, because the slow ones kept outliving the lease. A sixty-second lease fixed that but left a crashed job stalled for **59.7 seconds**. Both directions hurt."

"The fix is realizing the lease is measuring the wrong thing. It measures how long the work takes; I want to know whether the worker is alive. So the worker heartbeats, and the lease runs from the last heartbeat. Now a two-second lease gives zero duplicates *and* three-second recovery."

"But heartbeats have a hole, and it's the interesting one. A worker that's **paused** isn't dead. Its heartbeats stop, its lease expires, a second worker starts – and then the first one wakes up, with no idea any time passed, and writes its result. I measured that: **2.65% of jobs ended up holding the older worker's result**, and worse, the legitimate worker then got rejected because the run was already marked done. Two failures, no errors logged. The fix is a fencing token: every claim gets a strictly increasing number, and a write has to present the current one."

"Third, missed runs. Scheduler down six hours, jobs every five minutes – 14,400 missed. I expected a thundering-herd problem, measured it, and it drained in twelve minutes. **That wasn't the problem.** The problem is correctness. Run them all and a daily-digest job sends 2,880 duplicate emails. Skip them all and you lose 1,440 subscription billing periods – real money, silently. Run just the last one and you still lose the billing. **There is no correct global policy**, because the right answer depends on what the job *means*. So the policy is a value the job declares – all, skip, latest, or within-a-window – and the scheduler just calls it."

"Everything ties together through one key: `(jobId, scheduledFor)`. It comes from the schedule, not from a clock read or a UUID, so anyone can recompute it and get the same answer. That's what makes restart-dedupe work, what identifies a run across two workers, and what the job itself can hand to an email service as an idempotency key."

"For testing, six invariants and a fuzz – 2,000 randomized runs, zero violations. Two things worth mentioning. My first invariant was 'never two workers on one run', and it failed 1,848 of 2,000 trials. The code was right; **the invariant was wrong** – that's the thing I just said is impossible. And I mutation-tested: removing the fence check fails 152 of 400, removing dedupe fails 367. Removing backoff fails **zero** – because backoff is a load property, not a correctness property, so it belongs in the measurement, not the invariant. That same exercise also showed me a line I'd written 'to be safe' was doing nothing, so I deleted it."

16. Follow-ups – DAGs, ప్రాధాన్యత, సమయ మండలాలు

"ఒక job ఇంకొకటి పూర్తయ్యాక నడవాలంటే?"

అప్పుడు ఇది scheduler కాదు, **workflow engine** (Airflow, Temporal). తేడా: scheduler కి "ఎప్పుడు" తెలుసు; workflow engine కి "దేని తర్వాత" తెలుసు. Dependencies ఉంటే మీకు ఒక DAG, ఒక topological order, మరియు "ఒక node విఫలమైతే దాని కింది nodes ని ఏమి చెయ్యాలి" అనే కొత్త ప్రశ్న

వస్తాయి. ఇది ఈ doc lo చూసిన దాని కంటే పెద్ద problem – interview lo దీన్ని **స్పష్టంగా పరిధి బయట** పెట్టండి.

"కొన్ని jobs ముఖ్యమైనవి అయితే?"

#ready ని ఒక priority queue చేయండి. కానీ ఒక హెచ్చరిక: ప్రాధాన్యత ఉంటే **starvation** వస్తుంది – తక్కువ ప్రాధాన్యత jobs ఎప్పటికీ నడవకపోవచ్చు. పరిష్కారం: ఎదురుచూసిన సమయాన్ని బట్టి ప్రాధాన్యత పెంచడం (aging), లేదా ప్రతి స్థాయికీ కొంత భాగం కేటాయించడం. DD 22 §11 lo **LevelQueue** ఇదే ఆలోచన.

"రోజూ ఉదయం 9 గంటలకి" – ఎవరి 9 గంటలు?

ఇది ఊహించిన దాని కంటే చాలా కష్టం. every: DAY అనేది **తప్పు** – ఎందుకంటే DST lo ఒక రోజు 23 గంటలు, ఇంకొకటి 25 గంటలు. సరైన పద్ధతి: తదుపరి సమయాన్ని **స్థానిక క్యాలెండర్**లో లెక్కించి, తర్వాత UTC కి మార్చడం. మరియు రెండు నిజమైన అంచు పరిస్థితులు: DST వసంతంలో **2:30 AM ఉండదు** (ఆ job ని వదిలేయాలా, 3:00 కి నడపాలా?), శరత్తులో **2:30 AM రెండుసార్లు వస్తుంది** (రెండుసార్లు నడపాలా?). మీ **scheduledFor** UTC lo ఉంటే §12 key ఈ రెండింటినీ సరిగ్గా నిర్వహిస్తుంది.

"ఇది ఒక్క machine మీద నడుస్తోంది – అది పడిపోతే?"

#runs మరియు **lastScheduled** ఒక database lo ఉండాలి, memory lo కాదు. **claim()** ఒక atomic update కావాలి (UPDATE ... WHERE state='pending' RETURNING), లేకపోతే ఇద్దరు schedulers ఒకే run ని ఇస్తారు. **fence** ఒక database sequence కావచ్చు. ఈ doc lo **logic ఏదీ మారదు** – కేవలం **Map** స్థానంలో ఒక table వస్తుంది.

"Dead letter lo పడిన jobs ని ఏమి చెయ్యాలి?"

అవి **మౌనంగా పోకూడదు** – అదే **dead** స్థితి ఉనికికి కారణం. ఒక alert, ఒక పేజీ, మరియు వాటిని తిరిగి నడిపే ఒక button. 2,000 ప్రయోగాల్లో 6,279 runs dead letter కి వెళ్ళాయి; అవి కనిపించకపోతే అది 6,279 మౌన వైఫల్యాలు.

17. ఏమి నేర్చుకున్నాం

1. Exactly-once execution అసాధ్యం; exactly-once effect సాధ్యం. ఆగిపోయిన worker మన అడుపులో లేడు – కానీ అతను రాయబోయేటప్పుడు మన తలుపు తడతాడు, మరియు అక్కడ మనం "నువ్వు పాతవాడివి" అని చెప్పగలం. 29,841 సార్లు ఇద్దరు workers ఒకే పని చేశారు; ప్రతిసారీ ఒక్క ఫలితమే నిల్వ అయింది.

2. Backoff ఎదురుచూపుని పెంచుతుంది, ఎదురుచూసేవాళ్ళని విడదీయదు. Synchronized clients కి ఘాతీయ backoff **సరిగ్గా అదే వృథా** చేసింది – 1,840 పిలుపులు – కేవలం 22 రెట్లు ఎక్కువ సమయంలో. యాదృచ్ఛికతే వాళ్ళని విడదీస్తుంది.

3. Retry ని మెరుగుపరచడం కంటే retry అవసరాన్ని తొలగించడం మంచిది. Jitter వృథాని 1,840 నుంచి 708 కి తెచ్చింది. షెడ్యూల్నే పరచడం దాన్ని **0** కి తెచ్చింది.

4. Worker చావడం కంటే ఆగిపోవడం ప్రమాదకరం. చచ్చినవాడు తిరిగి రాడు. ఆగినవాడు తిరిగి వచ్చి, తనకు ఏమీ తెలియకుండా, సరైన ఫలితాన్ని తుడిచేస్తాడు – 2.65% సార్లు.

5. ఒక limit ని ఎంచుకునేటప్పుడు సగటు చూడకూడదు, తోక చూడాలి. Lease ని సగటు వ్యవధి బట్టి ఎంచుకుంటే 16.4% jobs రెండుసార్లు నడిచాయి. తోక limit ని నిర్ణయిస్తుంది.

6. కొన్ని ప్రశ్నలకి engineering సమాధానం ఉండదు. "తప్పిపోయిన runs ని ఏమి చెయ్యాలి" అనేది బిల్లింగ్ కి all , health check కి skip . Scheduler ఆ నిర్ణయం తీసుకోకూడదు – అది ఆ నిర్ణయాన్ని మోసుకెళ్ళాలి.

7. సమయం నుంచి వచ్చే key యాదృచ్ఛికత నుంచి వచ్చే key కంటే బలమైనది. (jobId , scheduledFor) ని ఎవరైనా, ఎప్పుడైనా, ఎన్నిసార్లైనా మళ్ళీ లెక్కించగలరు – మరియు అందరికీ అదే జవాబు వస్తుంది. Date.now() లేదా UUID అలా కాదు.

8. నియమం తప్పయితే code ని ఎంత వెతికినా ఏమీ దొరకదు. నా fuzz 2,000 lo 1,848 సార్లు విఫలమైంది – ఎందుకంటే నేను సాధించలేని దాన్ని పరీక్షిస్తున్నాను. Test విఫలమైనప్పుడు మొదట అడగాల్సిన ప్రశ్న "code lo ఏమి తప్పు" కాదు, "నేను అడుగుతున్నది సరైనదేనా".

9. రెండు రకాల లక్షణాలకి రెండు రకాల రుజువులు. Backoff ని తీసేసినా fuzz 0/400 చూపించింది – ఎందుకంటే backoff ఒక భారపు లక్షణం, సరి-తప్పు లక్షణం కాదు. దాని విలువ §4 పట్టికలో ఉంది. Mutation testing ఒక పంక్తి నిజంగా ఏమీ చేయడం లేదని కూడా చూపించింది, మరియు నేను దాన్ని తీసేశాను.

ఈ DOC నుంచి ఒక్క వాక్యం గుర్తుపెట్టుకోవాలంటే

Scheduler యొక్క ప్రతి కష్టమైన సమస్య "సమయం గడిచింది, కానీ ఎవరికి ఎంత గడిచిందో వేరు" అనే ఒక్క నిజం నుంచే పుడుతుంది.

అందరూ ఒకే క్షణం అనుకుంటారు (§4). Worker కి సమయం ఆగిపోయింది, మనకు గడిచింది (§8). Scheduler కి 6 గంటలు గడిచాయి, jobs కి అది ఇంకా రాత్రి 2 గంటలే (§10).

అందుకే పరిష్కారాలన్నీ ఒకే ఆకారంలో ఉంటాయి: సమయాన్ని ఒక అంచనాగా కాకుండా, ఒక నమోదైన వాస్తవంగా మార్చడం – scheduledFor , lastBeat , fence . మూడూ "ఇది ఎప్పుడు నిజమైంది" అని రాసి పెట్టుకునే మార్గాలే.